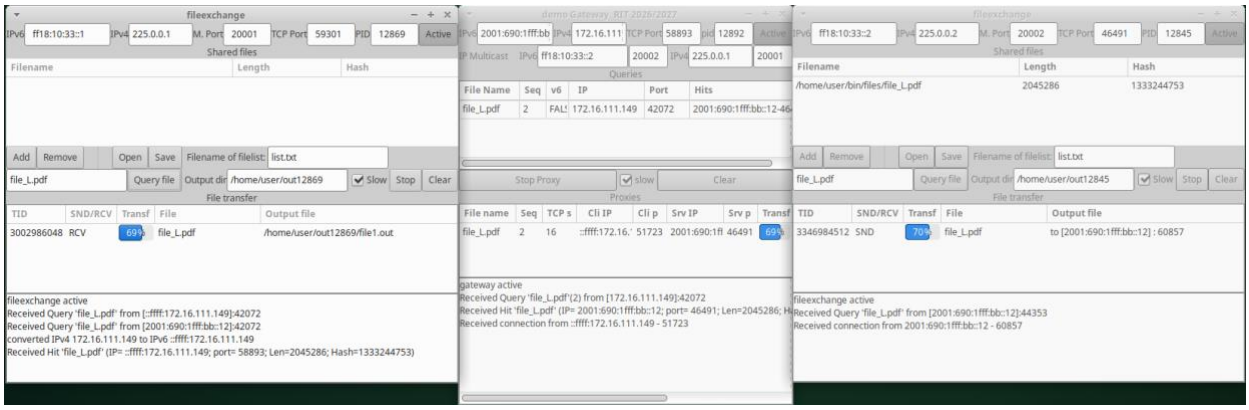
NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING



Integrated Telecommunication Networks (RIT)

2026/2027

2nd Laboratory Work:

Gateway application for file exchange between IPv4 and IPv6

Classes 7 to 10

Mestrado em Engenharia Eletrotécnica e de Computadores / MERSIM

https://tele1.deec.fct.unl.pt/rit

Luis Bernardo

1. Objectives

Familiarisation with programming using IPv4 and IPv6 addresses, the Gnome/Gtk+3 graphical environment, and the mechanisms of thread management and inter-thread synchronisation in the Linux operating system.
The work consists of developing a *gateway* for a file exchange application in a dual-stack network. The gateway registers with an IPv6 *multicast* address and an IPv4 *multicast* address and resends the query packets between these two addresses using a *unicast* IPv6 *socket*. It then propagates the responses back, allowing the search to run in parallel across the two client groups. To enable file transfer between IPv4 and IPv6, the application translates addresses in the response packets. From IPv6 to IPv4, it connects two TCP connections and acts as a proxy. Each connection between IPv4 and IPv6 TCP sockets runs in a POSIX (*pthread*) thread, allowing concurrent data transfer from multiple streams.

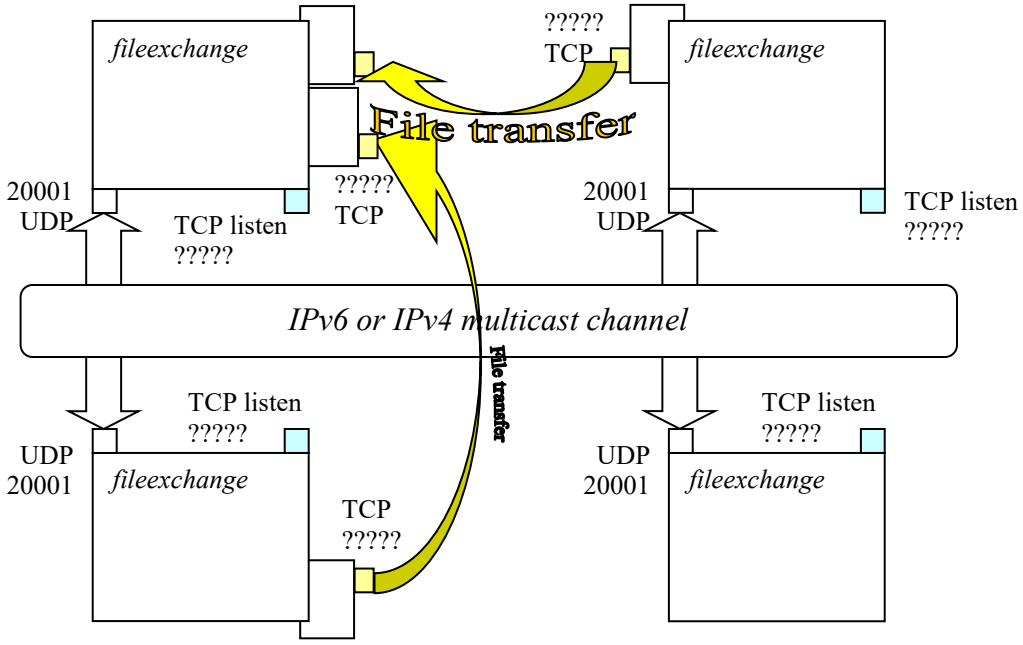
The work involves implementing a program: ***gateway***.

2. Specifications

The gateway application allows you to interconnect file exchange clients that operate in the IPv4 domain with clients that operate in the IPv6 domain, and vice versa.
In each domain, the *fileexchange* application is used, provided with the assignment, which supports searching the network and downloading the files located.

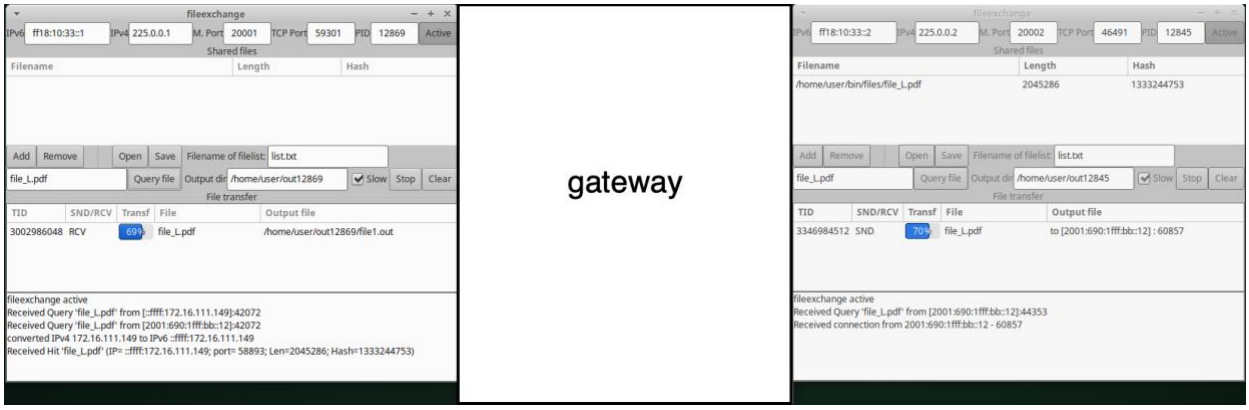
2.1. The *fileexchange* application

The *fileexchange* application performs file search using a datagram socket. To receive requests, it uses a UDP socket associated with the *multicast* addresses of an IPv4 group and an IPv6 group, configured for a single port number for both groups. To send requests and receive responses, it uses another private UDP socket (with a single port). To exchange files, it uses a TCP socket to receive connections (on a single port) plus an arbitrary number of TCP *sockets* to send or receive files on *concurrent* threads.



The *fileexchange* application starts by waiting for the user to configure the channel (IPv6 multicast address + IPv4 multicast address + port number) where they want to listen. After the

user presses a button, the application starts up with all *sockets* and is ready to send or receive UDP messages or TCP connections. At the same time, the application allows you to modify the list of shared files.

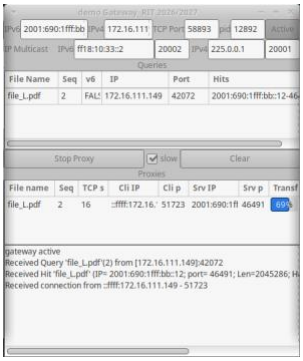


When the user chooses to search for a file, the client application sends the request to both groups and waits for a response up to the maximum time (5 s). After receiving a response, the client launches a *thread* for that connection, which creates a temporary TCP socket to receive the file. In the end, it saves the file in an output directory defined in the graphical interface.

When it acts as a sender, upon receiving a new TCP connection, the *fileexchange* application also creates a *thread* to send the requested file. The main window lists all active threads, received files, and the percentage of bytes transferred.

The *fileexchange* application has a *Slow* button that allows it to be intentionally slow, with a 0.5 second 'sleep' in the file send/retransmit/receive cycle, which makes it "sleep" between blocks of the file. This way, it allows you to test situations with several concurrent transmissions or transmission interruptions.

2.2. The *gateway* application

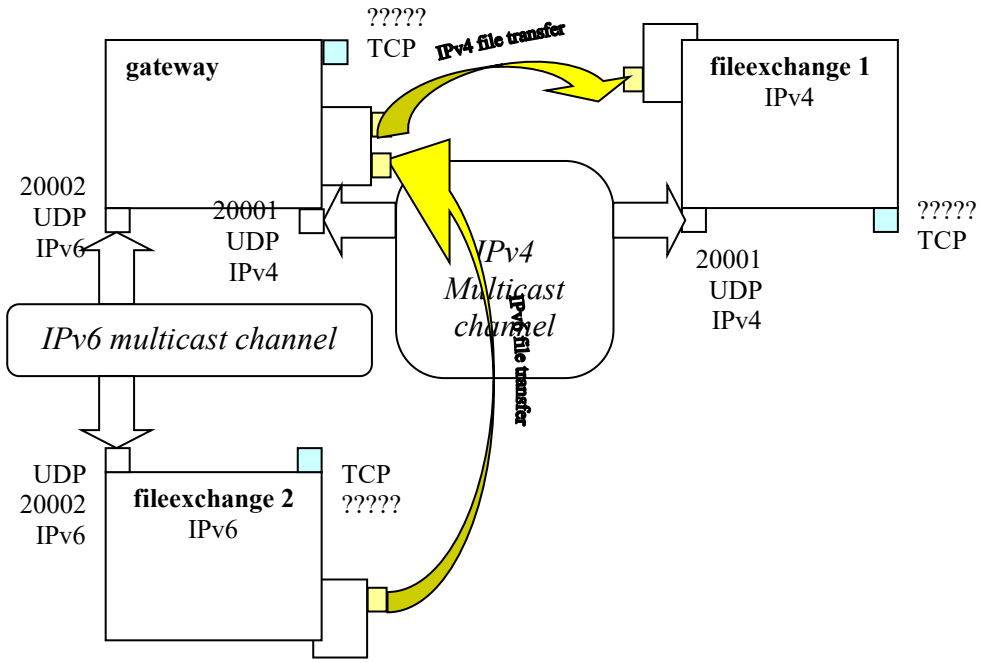


The *gateway application* must receive requests from each network and propagate them to the other, as long as they have not been propagated previously. To tolerate the existence of several *active gateways* at any given time, broadcasting on the other network must be preceded by a random delay. The *gateway* must create a table indexed by the active search key, defined by the set formed by the file name, a sequence number, and the network from which the search originates (IPv6 or IPv4). Responses must be propagated in reverse. **In the case of an IPv6-to-IPv4 lookup, the IPv4 addresses of the servers shall be translated into the dual ones in the form *"::ffff:IPv4"*.** In an IPv4-to-IPv6 lookup, the *gateway* shall exchange the addresses and ports, respectively, for the local IPv4 address and TCP port of the server socket created specifically for the request, to which the *gateway* accepts connections.

In this second scenario (IPv4 to IPv6), the *gateway* will receive TCP connections from clients and will establish a connection to the remote IPv6 client, acting as an intermediary.

The *gateway* application requires four main configuration parameters: the IPv4 and IPv6 *multicast addresses* of the groups (by default, the addresses "225.0.0.1" and "ff18:10:33::2" will be used) and the UDP port numbers for IPv4 and IPv6 (by default, ports 20001 and 20002 will be used, respectively). The suggestion is to have one fileexchange 1 application configured with the addresses "225.0.0.1" and "ff18:10:33::1" and port 20001, and a second fileexchange 2 application configured with the addresses "225.0.0.2" and "ff18:10:33::2" and port 20002. By running *the fileexchange* 1, *fileexchange* 2, and *gateway* applications on the same network, you can make transfers from IPv4 to IPv6 when making a request from the *fileexchange* 2

application, or from IPv6 to IPv4 when making a request from the *fileexchange* 1 application. The requested filenames must not exist locally.

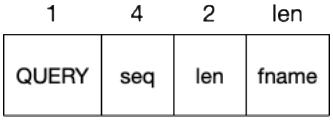


To facilitate the development of the work, an Eclipse project is provided with the interface specification file "*gateway.glade*" with the main window of the program, plus a set of functions for accessing the graphical interface, interacting with *sockets* and reading and writing control packets. These files already include the reading of the configuration parameters of the *multicast sockets* and the skeleton of the *callback functions* for UDP and TCP data (which process the requests).

2.3. File search protocol

From the moment it is active, the *fileexchange* application can send a file search message to the groups. The query message (*Query*) sent to the group consists of an octet with the message type, followed by a unique sequence number and the length and string with the name you want to search. The name must not include the path (directory names) and must be terminated with the character '\0'. It is possible to run multiple searches in parallel using different sequence numbers.

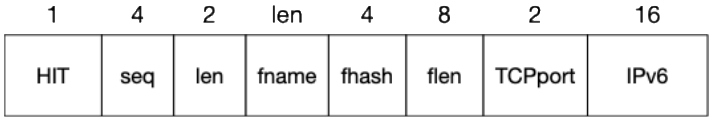
```
File search message: { sequence of }
 unsigned char type;          // message type  -  QUERY= 20
 uint16_t seq;                // request sequence number
 short int len;               // length of 'fname' string = strlen(fname)+1
 char[] fname;                // 'fname' string
```



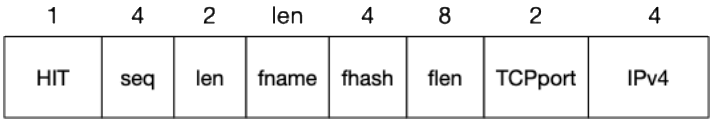
The hit message sent to the sender of the query (*Hit*) consists of an octet with the type of message, followed by the sequence number sent by the searcher, the length and name of the requested file, the byte length of the file, the *hash value* of the file's contents, the port of the *TCP socket* where it receives connections, and the IP address of the server. An IPv6 address is returned in an IPv6 network, while an IPv4 address is returned in an IPv4 network.

```
File hit message: { sequence of }
 unsigned char type;          // message type  -  HIT= 10
 uint32_t seq;                // request sequence number
 short int len;               // length of 'fname' string = strlen(fname)+1
 char[] fname;                // 'fname' string
 uint32_t fhash;              // File content's hash value
 unsigned long long flen;     // File length
 unsigned short TCPport;      // server's TCP port number
 union {
  struct in6_addr IPv6;       // server's IPv6 address
  struct in_addr IPv4;        // server's IPv4 address
 } u;
```

IPv6 network:



IPv4 network:



The *gateway* application must memorize all incoming query messages and the respective addresses and ports of the senders, keeping this information for at least 10 seconds, in order to know which queries have appeared in each of the networks and, later, forward the hit messages. If it is a new query, the gateway must send it to the other network, introducing a random delay, in order to tolerate the existence of several *gateways* connecting two networks. If it does not receive a hit, it must abort the session associated with the query. The random delay can be calculated using the *random* function:

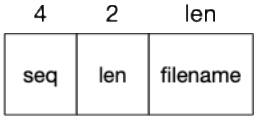
```
long int jitter_time= (long)floor(1.0*random()/RAND_MAX*QUERY_JITTER);
```

Upon receiving a Hit message, the gateway application must validate that the Query still exists in its list of resent query messages. It must then translate the hit to a valid one on the destination network:

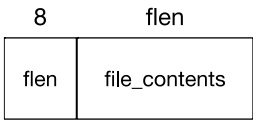
- In the IPv6 network, the gateway must modify the received IPv4 address to the equivalent IPv6 address (*::ffff:IPv4*), leaving the dual protocol stack solve the problem of the destination-to-destination TCP connection (it is assumed that all nodes have dual stack);
- In the IPv4 network, the gateway must change the received IPv6 address to the IPv4 address of the *gateway*, and replace the port number with the number of the TCPv4 socket where the *gateway* receives connections. Simply memorize the first received `Hit`, which is then used when receiving the TCP connection from the clients to establish the connection to the server.

2.4. File transfer

After receiving the first hit message, the *fileexchange* client application (file receiver) downloads the file, ignoring the following hits received. It initiates a TCP connection to the file sender (which sent the *Hit*). On this channel, communication begins with the sending of a file request header. The receiver tells the sender the file name and sequence number previously sent during the search. The sender in response starts by sending the file length (or zero, if it doesn't exist) and then sends the file contents.



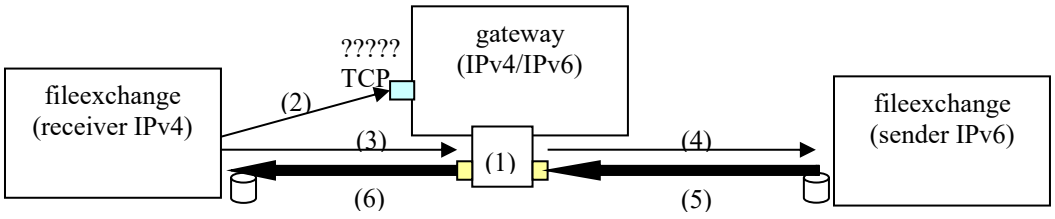
```
File request: { sequence of }
 uint32_t seq;      // request sequence number
 int16_t len;       // Filename length(max. 256 bytes)
 char[] filename;   // Filename string ended by '\0'

Reply to file request: { sequence of }
 unsigned long long flen;// File length (0 if non existing)
 char[] file_contents;   // File contents
```



When the request comes from the IPv6 network, the *gateway* application does not participate in the file transfer. The TCP connection is established directly between the two *fileexchange* applications.

When the request comes from the IPv4 network, two TCP connections are created: one on the IPV4 network to the *gateway* and the other from the *gateway* on the IPv6 network. The sequence of events is represented in the following figure. The TCP server socket is created at the start of the application (1), at which point a connection reception function (the `handle_new_connection` callback) is called. When it receives a connection (2), it creates a TCP socket (with *accept*) to communicate with the receiver on the IPv4 network, starting by receiving the request header (3). After receiving the request, it creates a new TCP socket that will connect to the sender through the IPv6 network and send the request (4). It then echoes the reply header and data (5)(6) that it receives from the sender, until the entire file is transmitted.

However, it is intended that the *gateway* does not have a merely passive role, but that it monitors the communication, validating the messages exchanged, and indicating the % of the file that has already been sent. For example, after 10 seconds waiting to receive data at one socket, you should consider that the connection has failed, and drop the file transfer.



It is also intended to optimize the throughput of the *gateway* in file exchange and to allow the concurrent sending of multiple files. Using *pthreads*, each file transfer thread will run concurrently, which requires that the state of the file transfers and *pthreads* be stored in dynamically allocated data structures, so that each thread has an independent state, in addition to the local variables of the *proxy_function* function.

3. Program development

3.1. Application installation

The gateway application project can be installed from *github.com* by cloning the project https://github.com/lflbernardofct/gateway_rit.

The gateway application project can also be obtained by downloading from https://tele1.deec.fct.unl.pt/rit a VMware virtual machine running Xubuntu 24.04, the Eclipse project, and the Eclipse development environment. In this case, you must first install VMware Fusion (for MacOs) or VMware Workstation Pro (for Windows). You need to register on the Broadcom support page before you can download the application.

As in the first lab work, you will also need to use the **git** version management system, to save intermediate versions of the project. On Linux it is simple to use *git* through the command

line, and you can find a summary of the most important commands at https://education.github.com/git-cheat-sheet-education.pdf.

git can also be used through Eclipse, or in another IDE (*Integrated Development Editor*). In the case of Eclipse, you can consult the following tutorial https://www.geeksforgeeks.org/git/how-to-use-git-with-eclipse/ .

3.2. Code provided

To facilitate the development of the application, we provide a fully functional test program (*demo_gateway* + *demo_gateway.glade*), the *gateway.glade* file with the definition of the graphical interface of the test program represented below, plus the set of files described below.

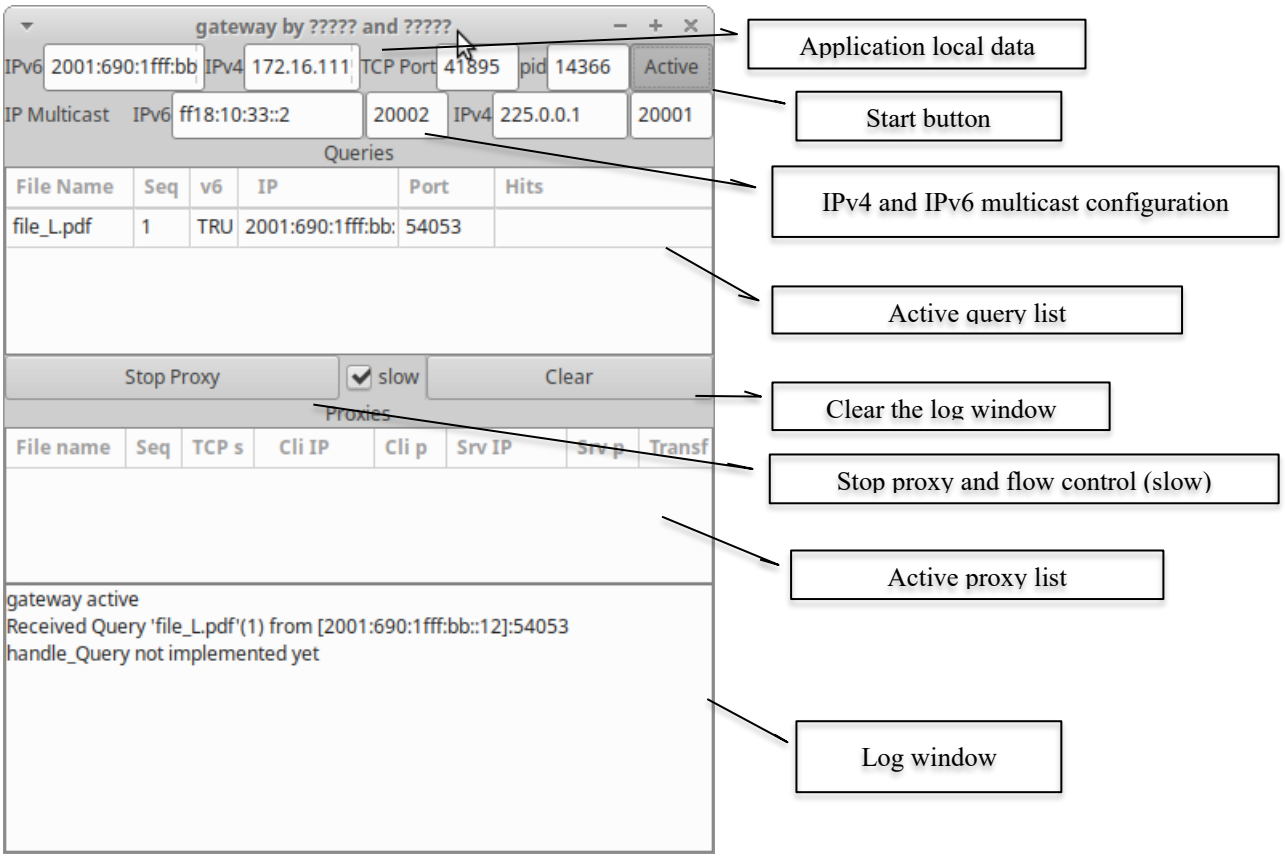
The interface contains a first line where you write the local application data (IPv6 and IPv4 addresses, the TCP port number where it receives connections from a proxy, and the process id number). The "*Active*" button controls the gateway startup. The second line allows you to configure the IPv6 and IPv4 multicast addresses and port numbers.

The fourth row contains a table (of type *GtkTreeView*) with the list of queries (*Query*) received that are waiting for Hits or have active TCP connections. The table represents the name of the requested file, the sequence number, the source of the request (IPv6 or IPv4), the IP address and port of the socket from which the request was received, and the *Hits* received.

The next line contains the commands to stop a proxy thread (*Stop Proxy*), control the pace of data sending (*Slow*) and clear the *log window*. The line with the proxy to be deleted must be marked in advance before pressing the button to stop it.

The sixth line contains the list of active proxies at a given point in time, with the file name, sequence number, the TCP port number of the server socket allocated to the request, the IPs and ports of the sockets of the receiving and sending *fileexchanges*, and the percentage of bytes transferred.

The seventh and final line contains a box for writing messages.



The program provided is composed of six modules:

- *sock.{c,h}* – Function library for handling *sockets* and IP addresses. **Complete**;
- *gui.h* e *gui_g3.c* – Library of functions to handle the graphical elements of the GUI, including the query and proxy lists. **Complete**;
- *callbacks_socket.{c,h}* – Socket start and stop functions, socket interaction callbacks, and UDP message encoding. **Incomplete –** the *write_query_message* function that encodes Query messages in a buffer is still to be programmed ;
- *callbacks.{c,h}* – Application start and stop functions, *timers*, and program logic control. **Incomplete** – most of the functions are still to be completed;
- *proxy_thread.{c,h}* – Functions that implement the proxy functions using a *pthread*. **Incomplete –** The thread's function and the state and socket management are still to be completed;
- *main.c* – Application startup function. **Complete**;
- *Makefile* – Compilation control script. It can be used to define symbols (e.g., DEBUG), control the level of debugging in the compiler, or enable debugging modules such as **sanitize**, which allows detecting runtime violations in memory access and memory leaks. **Complete**.

The provided program includes all files except *the proxy_thread files.{c,h}*, *callbacks.{c,h}* and a small part of *callbacks_socket.{c,h}*, which must be completed by the students. In multicast communication, this includes preparing the QUERY message content for sending, configuring the timers, and processing the QUERY and HIT messages. For file communication through TCP sockets, the protocol still needs to be programmed, along with additional mechanisms to handle error situations.

The initial part of the work consists of reading and understanding the provided code to make the UDP communication (Phase 1) feasible in a short time. It is recommended that students, BEFORE THE FIRST CLASS, LOOK AT THE CODE PROVIDED IN DETAIL to get the most out of it. Several comments are included to help you understand the logic of the provided code.

3.3 Goals

The work should be developed in three distinct phases, each with several tasks. **At the end of each phase, you should save a checkpoint in git with the names *Phase1*, *Phase2* and *Phase3*.**

Phase 1

The first phase supports basic file searches between the IPv6 and IPv4 domains (IPv6 Query and IPv4 Hit), without fail-safe mechanisms. The tasks of Phase 1 are:

1.1. Finish programming the `write_query_message` function (*callbacks_socket.c*) in order to prepare a buffer with the message content from the received arguments;

1.2. Program the "Query" package handling function (`handle_Query`, *callbacks.c*) (the packet is already read in the provided code) in order to resend the message in the other domain. You should complete the structure definition (`struct Query`, *callbacks.h*) to remember the addresses from which the Query came, and update the constructor function (`new_Query`, *callbacks.c*) to save that information in a list. You should also send the received IPv6 message to the IPv4 domain using the `send_multicast function`. NOTE: If you have a lot of difficulty using the provided list, you can use the query graphical list to accomplish this task (with a slight discount on the final grade);

1.3. Program the HIT message handling function (`handle_Hit`, *callbacks.c*) to receive hits from IPv4 (the packet is already read in the provided code). This function must send the HIT messages to IPv6, converting the IPv4 address of the *fileexchange* with the requested file to the equivalent IPv6 address.

1.4. git: create checkpoint "Phase1"

Phase 2

The second phase aims to support searching for files between the IPv6 and IPv4 domains (IPv6 Query and IPv4 Hits) with fault-tolerance mechanisms. The tasks of Phase 2 are built, in part, by redoing the same functions as in Phase 1, adding several timers, all processed in the function (`callback_query_timeout`, *callbacks.c*), and a state to know what is the next event expected. The tasks of Phase 2 are:

2.1. Extend the structure (`struct Query`, *callbacks.h*) to include a state variable, which allows you to remember which stage of a request's processing process it is in (e.g., which timer is active);

2.2. Set a timer to remove a Query from the list when a Hit is not received after 10 seconds;

2.3. Set a timer to keep the contents of the Hit visible in the GUI for 10 seconds after the hit message is received;

2.4. Set a timer with a random time to resend the *Query packet* on the other network (on `handle_Query`), to avoid duplicate sending of queries on a network when there are multiple *gateways* active between two networks;

2.5. git: create checkpoint "Phase2"

Phase 3

The third phase aims to support the search and transfer of files between the IPv4 and IPv6 domains (IPv4 Query and IPv6 Hit). File transmission is done through an application *gateway* that connects two TCP tunnels, using a *thread* that relays the file contents.

The Phase 3 tasks will complement some of the Phase 2 tasks and introduce threads for file transmission in file *proxy_thread.c*. The Phase 3 tasks are:

3.1. Program the HIT message callback function (`handle_Hit`) to process hits from IPv6. In this case, you must return a HIT pointing to the gateway's TCP socket, and you can add new statuses to record that you are waiting to receive a new TCP connection for a QUERY or the evolution of communication on TCP connections;

3.2. Set a timer to remove a QUERY when the maximum time expires without receiving a TCP connection;

3.3. Program the `proxy_function` function (*proxy_thread.c*), to perform the sequence of operations, from receiving and sending the request and response headers, to sending the file contents. The type and status-related functions are defined in the *proxy_thread.h* and *proxy_thread.c* files, respectively. You should also update the received file percentage in the GUI;

3.4. Configure the TCP *sockets* to limit the wait time for read operations and to maximise the throughput reached for each connection;

3.5. git: create checkpoint "Phase3"

This lab work is much more demanding in terms of programming than the first one. In order to get to the end of the four weeks of work with everything ready, it is necessary to use all the practical classes, and you must: finish task 1.2 at the end of the 1st week of work; finish phase 2 (task 2.5) after the 2nd week; be working on task 3.3 at the end of the 3rd week; finish the work

as complete as possible in the fourth and last week. **At least, all students must reach the end of task 2.3.**

Do not forget that at the end of the semester it is necessary to turn in ALL the works. Do not leave for the last week what you can do over the first four weeks, because **YOU WILL NOT BE ABLE TO DO ALL THE WORK IN THE LAST WEEK!**

STUDENT POSTURE

Each group should consider the following:

- Don't waste time on data input and output aesthetics
- Program according to the general principles of good coding (use of indentation, comments, use of variables with names that conform to their functions ...)
- Ensure that the work to be done is equally distributed among the two group members
- The use of artificial intelligence tools in developing the code is allowed. The final oral discussion will assess the students' level of knowledge about the work delivered. It is better to deliver a more incomplete work, which you know well, than a very complete one that you do not know, which can lead to not being approved in the curricular unit.